

Nature of Software Engineering Economics (SEE)

A decorative graphic consisting of a 3x3 grid of squares. The center square is dark blue with the number '11' in white. The surrounding squares are light blue, with the top and bottom center squares being white.

11

Lesson

Outline

1. Definition of Software Engineering Economics (SEE)
2. Decision Making process of SEE
3. SEE KAs & KPs in IEEE SWEBOK2014 / C-SWEBOK2019
4. Principles of SEE
5. Benefits of Learning SEE

11.1 Introduction

■ Case Study

Top 10 companies by market valuation in the world in 2007-2024						Unit: Billion in USD
Year	2007		2020		2024	
Rank	Name	Market Cap	Name	Market Cap	Name	Market Cap
#1	ExxonMobil	467	Apple	1576	Microsoft	3100
#2	General Electric	394	Microsoft	1551	Apple	2680
#3	Microsoft	265	Amazon	1433	Nvidia	2210
#4	ICBC	259	Alphabet (Google)	980	Saudi Aramco	2010
#5	Citibank	243	Facebook	676	Alphabet (Google)	1840
#6	AT&T	238	Tencent	621	Amazon	1810
#7	Royal Dutch Shell	232	Alibaba	580	Meta Platforms	1260
#8	American Bank	230	Berkshire Hathaway	433	Berkshire Hathaway	884
#9	China Petroleum	225	VISA	413	Eli Lilly	725
#10	China Mobile	207	Johnson&Johnson	371	TSMC	709

Question: What issues can you observe from this case ?

What are the reasons for these issues ?

11.1 Introduction

■ Reason: Friction-free

If you are not familiar with the term, friction-free refers to a product or service that is easy to buy, consume, and switch to.

The information highway will extend the electronic marketplace and make it the ultimate go-between, the universal middleman This will carry us into a new world of low-friction, low-overhead capitalism, in which market information will be plentiful and transaction costs low. It will be a shopper's heaven.

In his 1995 book “**The Road Ahead**” Microsoft CEO Bill Gates writes: Information about venders and their products will be available to any computer connected to the highway. Servers distributed worldwide will accept bids, resolve offers into completed transactions, control authentication and security, and handle all other aspects of the marketplace, including the transfer of funds.

Example: Information transfer

Beacon tower → Fast horse + Post station → Telegraph & Telephone →
Satellite Communication → Internet

11.1 Introduction

■ What do we learn from this case?

This case study illustrates the variety of issues with which managerial economics is concerned.

The following questions arise:

1. Why are software/information service companies developing rapidly?
2. Are there solutions to the problem, in terms of strategy/action that can be taken?
3. What constraints or conditions exist in terms of operating any strategies?
4. How do we evaluate these strategies in terms of costs and benefits?
5. What are the effective ways of measuring the relevant variables of costs?
6. What assumptions should be made in our analysis of software engineering?
7. How do we deal with the problem of risk and uncertainty regarding to the effects of strategies of software project/product in the future?
8. What criteria can we use for selecting the better strategies and taking action?

Software engineering economics will provide you the powerful methods/tools to solve above questions.

11.2 Definition

■ Definition of Software Engineering Economics (SEE)

Software Engineering Economics

The application of economic theory and methods to software engineering business decision-making. As such it can be seen as a means to an end by managers, in terms of finding the most efficient way of allocating their scarce resources, optimizing business model & design, verifying and achieving their goals.

Business (业务)

The term ‘business’ must be defined very broadly in this course. It applies to any situation where there is a transaction between two or more parties/stakeholders.

Related theoretical foundations

The software engineering economics involves the following theories:

1. Theory and Practices of Software Engineering (base)
2. Economic Theory (base)
3. Decision Science (tools)
4. Business and Accounting theory (context and metrics)
5. Project Management (process)

11.2 Definition

■ Economic Theory

Assumption

This is essentially an approach that treats the individual elements within the economy (consumers, firms and workers) as rational agents with objectives that can be expressed as quantitative functions (utilities and profits) that are to be optimized, subject to certain quantitative constraints.

Essentially **Positive** statements are factual statements whose truth or falsehood can be verified by empirical study or logic. **Normative** statements involve a value judgement and cannot be verified by empirical study or logic.

Example: Positive vs Normative statements

1. The distribution of income in China is unequal (不公平).
2. The distribution of income in China is inequitable (不公正).

The first statement is a positive one while the second is a normative one. Normative statements often imply a recommendation, in the above example that income should be redistributed. For that reason they often involve the words *ought* or *should*.

11.2 Definition

■ Decision Science

The decision sciences provide the tools and techniques of analysis used in decision making.

The most important aspects are as follows:

- Numerical and algebraic analysis
- Optimization
- Statistical estimation and forecasting
- Analysis of risk and uncertainty

These tools and techniques will be introduced in the following lessons.

11.2 Definition

■ Business Theory

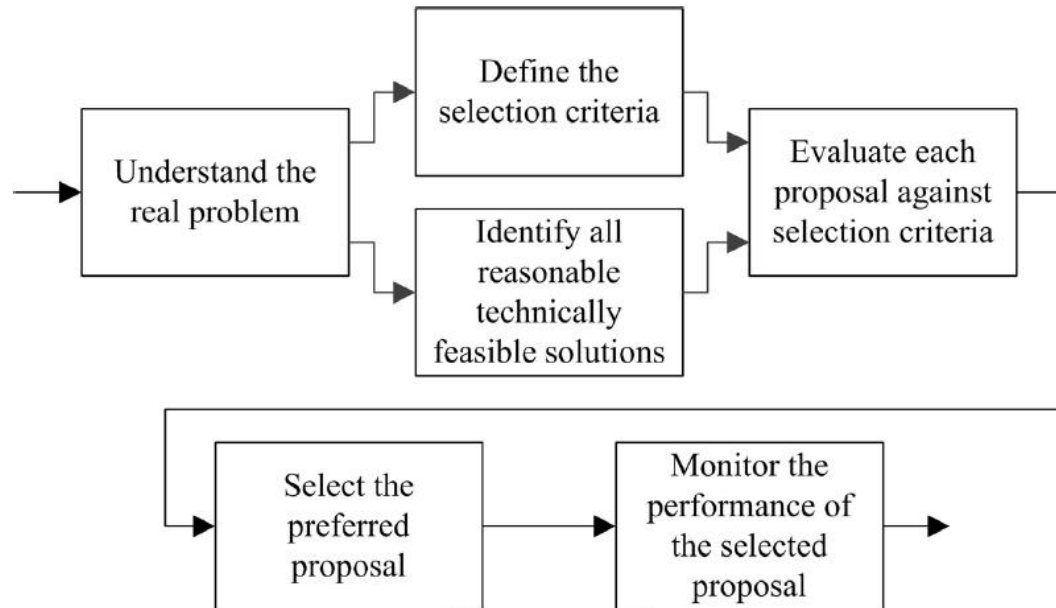
All firms consist of organizations that are divided structurally into different departments or units, even if this is not necessarily performed on a formal basis. Typically the units involved are:

- Production and operations
- Marketing
- Finance and accounting
- Time value of money
- Human resources management

All of these functional areas can apply the theories and methods mentioned earlier, in the context of the particular situation and tasks that they have to perform.

11.3 Decision-Making Process of SEE

■ “Problem-Modelling-Solving” Model



Comparison:

“**How to solve it**” proposed by Polya in software engineering courses

1. Understand the problem (communication and analysis).
2. Plan a solution (modeling and software design).
3. Carry out the plan (code generation).
4. Examine the result for accuracy (testing and quality assurance).

11.4 KAs & KPs of SEE

■ KAs & KPs in SWEBOK

Software Engineering Body of Knowledge

IEEE SWEBOK version3, 2014

Detailed contents of SWEBOK can be found in the folder [ReferenceMaterials](#) on Canvas platform.



11.5 KAs & KPs of C-SWEBOK

■ KAs & KPs in C-SWEBOK

Chinese Software Engineering Body of Knowledge

Working Group of C-SWEBOK of MOE, C-SWEBOK, 2019

Detailed contents of SEE can be found in the folder [ReferenceMaterials](#) on Canvas platform.



图 14-1 软件工程经济学知识领域知识单元分解图

11.6 Principles of SEE

■ Three Principles of SEE

Principle One: Allocation

- allocate all of resources needed in your software project.

Principle Two: Optimization

- optimize the goal, strategies, processes and activities of your software project.

Principle Three: Verification & Validation

- verificate and validate the plan, actions and deliverables of the whole project or each stages.

Example

Agile methods commonly used in software process.

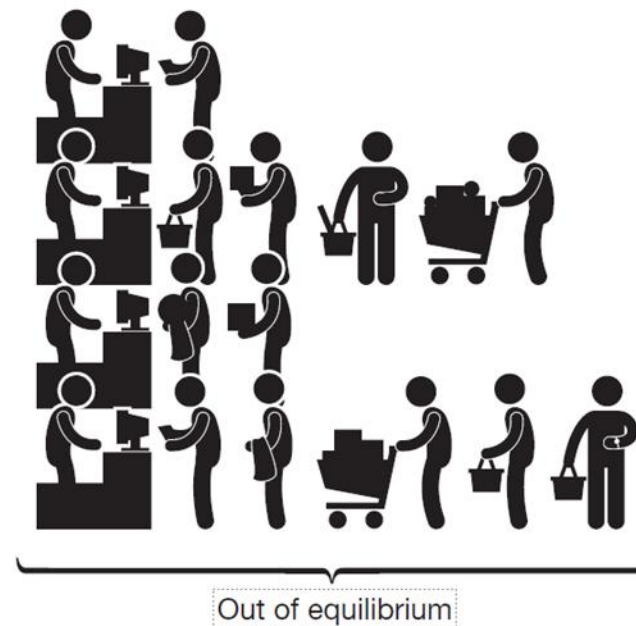
Please thinking: what principles applied in agile methods?

11.6 Principles of SEE

■ Allocation of resources

In most situations, you are not the only man in software project. Other people's behavior will influence what you decide to do. Economists think of the world as a large number of economic agents who are interacting and influencing one another's efforts at optimization.

Equilibrium (均衡) is the special situation in which everyone is trying to optimize, so nobody believes that they would benefit personally by changing his or her own behavior.



11.6 Principles of SEE

■ Optimization

Optimization is the most important principle of SEE.

Software engineers believe that optimization — trying to pick the best feasible option — explains the choices that people make, including minor decisions like accepting an invitation to see a movie and major decisions like deciding whom to marry. Decisions aren't made with a crystal ball. People try to do as well as they can, given the limited information, knowledge, experience, and training that they have.

For example, Algorithm optimization refers to the optimization of the relevant performance of the algorithm, such as time complexity, space complexity, correctness and robustness. Because the application scenario of the algorithm changes greatly, the algorithm optimization can make the algorithm have the ability to better meet the needs of users.

11.6 Principles of SEE

■ Optimization

Example: Efficiency comparison of sorting algorithms

排序方法	时间复杂度（平均）	时间复杂度（最坏）	时间复杂度（最好）	空间复杂度	稳定性
插入排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
希尔排序	$O(n^{1.3})$	$O(n^2)$	$O(n)$	$O(1)$	不稳定
选择排序	$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$	不稳定
堆排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(1)$	不稳定
冒泡排序	$O(n^2)$	$O(n^2)$	$O(n)$	$O(1)$	稳定
快速排序	$O(n\log_2 n)$	$O(n^2)$	$O(n\log_2 n)$	$O(n\log_2 n)$	不稳定
归并排序	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n\log_2 n)$	$O(n)$	稳定
计数排序	$O(n+k)$	$O(n+k)$	$O(n+k)$	$O(n+k)$	稳定
桶排序	$O(n+k)$	$O(n^2)$	$O(n)$	$O(n+k)$	稳定
基数排序	$O(n*k)$	$O(n*k)$	$O(n*k)$	$O(n+k)$	稳定

11.6 Principles of SEE

■ Optimization

Case study The birth of compiler (IBM 1950')

FORTRAN language is a widely popular program language suitable for scientific computing in the world. It is also the earliest high-level programming language in the world. FORTRAN is the abbreviation of formula translation, that is, mathematical formula translator.

Case study Unbundling of software and hardware (IBM, 1969)

In 1969, IBM announced the unbundling of system/360 application software and mainframe hardware. IBM System/360 mainframe, which was introduced by IBM in 1964, is a general-purpose computer system. Its business goal is that users can continuously upgrade the host hardware according to the growing demand of their business transaction ability without replacing the host application program. This provides the technical feasibility of separating software from hardware.

In order to resist the competition of RCA, IBM decided to decouple the host application software from the mainframe. IBM announced that its software and hardware are unbound. This has brought the birth of the software industry.

11.6 Principles of SEE

■ Optimization: Trade-offs and Budget Constraints

All optimization actions involve trade-offs(权衡).

Trade-offs arise when some benefits must be given up to gain others. Think again about the simple case of Facebook. If you spend an hour on Wechat, then you cannot spend that hour doing other things.

For example, you cannot work at most part-time jobs at the same time you are editing your Wechat profile. (However, some students believe that they can read their friends' posts while listening to a lecture. When the teacher of this course try to do this myself, I find that I don't learn very much from that lecture).

Exercise:

Please design your cases using trade-offs in your course project.

11.6 Principles of SEE

■ Optimization: Trade-offs and Budget Constraints

Economists use budget constraints to describe trade-offs.

A **budget constraint** (预算约束) is the set of things that a person can choose to do (or buy) without breaking her budget.

Suppose: Only one of two activities with each hour of your free time:

— use social media.

— work at a part-time job.

Suppose that you have 5 free hours in a day (once we take away necessities like sleeping, eating, bathing, attending classes, doing problem sets, and studying for exams). Think of these 5 free hours as your budget of free time. Then your budget constraint would be:

$$\mathbf{5 \text{ hours} = \text{Hours on social media} + \text{Hours working at part-time job}}$$

This equation, which is also called a budget constraint, implies that you face a trade-off.

11.6 Principles of SEE

■ Optimization: Cost-Benefit Analysis

Cost-benefit analysis is a calculation that identifies the best option by summing benefits and subtracting costs, with both benefits and costs denominated in a common unit of measurement, like Yuan(RMB).

Cost-benefit analysis is used to identify the alternative that has the greatest net benefit, which is the sum of the benefits of choosing an alternative minus the sum of the costs of choosing that alternative.

Example: Trip planning from Shanghai to Beijing

— By Car driving, by train or by air ?

To analyze this problem using cost-benefit analysis, you need to list all benefits and costs of driving compared to the alternative of flying. Here we'll express these benefits and costs comparatively, which means the benefits of driving compared to flying and the costs of driving compared to flying. We'll need to translate those benefits and costs into a common unit of measurement.

What is your option?

11.6 Principles of SEE

■ Verification & Validation

Software development, like other industries, needs the continuous input of production factors before there is output. In addition to considering the balance of supply and demand (Equilibrium), how to distribute output among market participants is also an issue that needs to be considered in economics.

Economists test their ideas with data. We refer to such evidence-based analysis as empirical analysis or empiricism. Economists use data to determine whether our theories about human behavior — like optimization, equilibrium and allocation — match up with actual human behavior.

Of course, we want to know if our theories fail to explain what is happening in the world. In that case, we need to go back to the practice and come up with better theories. That is how software engineering economics progresses in general.

But there are some cases when cause and effect are hard to untangle. Does being relatively smart cause people to go to college, or does going to college cause people to be relatively smart? Perhaps both directions of causation apply.

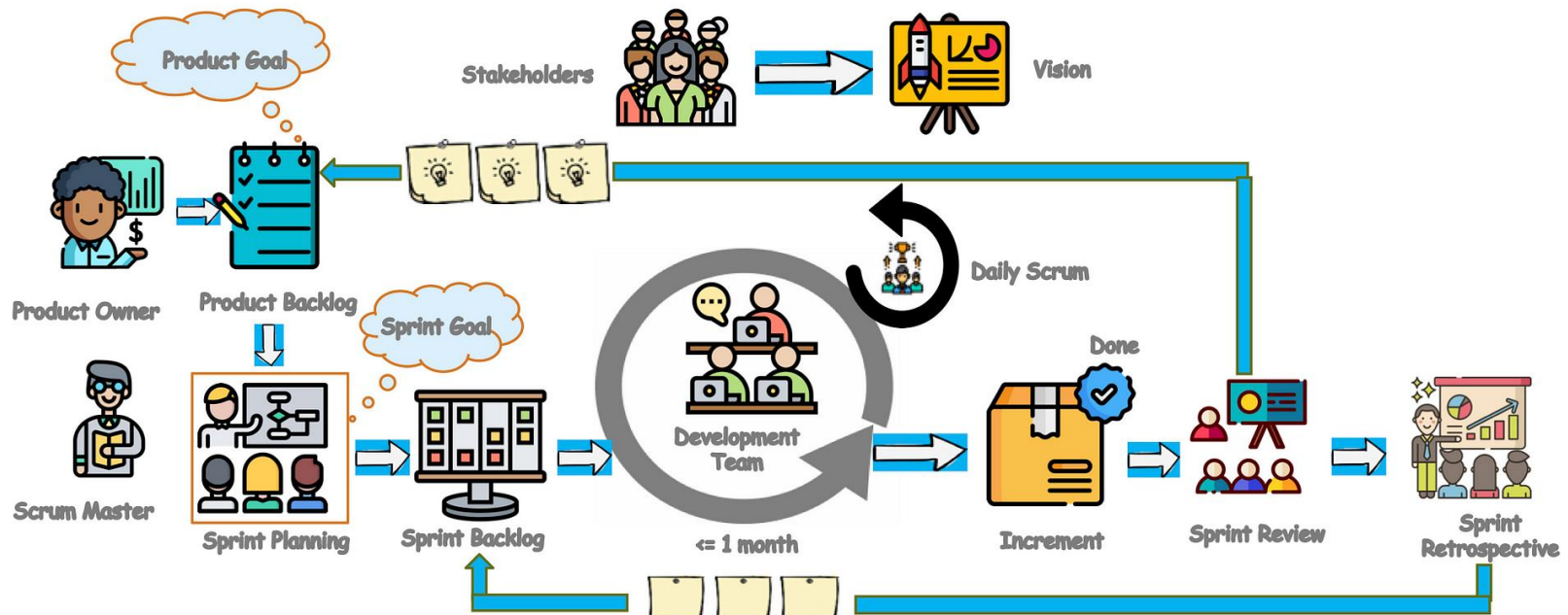
11.6 Principles of SEE

■ Exercise in class: Scrum

Scrum is a broadly used agile model in software engineering process.

Discussion: In this model, what principles of SEE are used in software activities such as task allocation, personnel configuration, software activities planning-monitoring-control, and deliverables/results verification and validation ?

Scrum Framework



11.7 Benefits of learning SEE

■ Benefits of learning SEE

Learning how to make good choices is the biggest benefit that you'll realize from learning economics.

Most decisions are guided by the logic of costs and benefits. Accordingly, you can use positive economic analysis to predict other people's behavior. Economics illuminates and clarifies human behavior.

Once you realize that you are constantly making economic choices, you'll understand that the learning of this course is only your first step. You'll discover the important cases or applications out of class.

The methods and tools of economics will improve your thinking and performance in all kinds of situations —making you a better engineer, a better consumer, and a better citizen.

Keep your eyes open and remember that every choice is economics in action.

Core Concepts

Software Engineering Economics

Positive and Normative statement

SEE Decision-Making Process

Optimization

Allocation

Verification

Trade-off

Opportunity Cost

Cost-Benefit analysis

Assignment

Join & Learn Software Engineering Economics(软件工程经济学) MOOC in Wisdom Tree(智慧树) MOOC platform, the linkage is

<https://coursehome.zhihuishu.com/courseHome/1000050734#teachTeam>

Please choose “Credit course”.

Notice

In the courses on **11th May**, we will carry out laboratory projects in class.

Please be sure to participate in these lab projects, write a experimental report after completing the relevant experiment, and submit the experimental report (PDF version) to Canvas.